

Tutorial 1 | OS Monsoon'25

Section A

Author → Rachit Bhandari



1. Linux

1.1 What is Linux?

Linux is an open-source Unix-like operating system kernel. It is the foundation of many distributions (Ubuntu, Debian, Fedora, Arch) that provide complete OS environments.

Why use Linux?

- Direct exposure to kernel-level operations. (operations which are very close to the hardware, or, low level system operations).

- Access to process management, memory handling, and system calls.
- Open-source flexibility for experimentation.

1.2 Linux Filesystem Overview

Directory	Purpose
/	Root of the filesystem.
/home	User home directories.
/bin	Essential binaries.
/etc	Configuration files.
/var	Variable data (logs, caches).
/tmp	Temporary files.
/proc	Virtual filesystem for process/kernel info.
/dev	Device files.

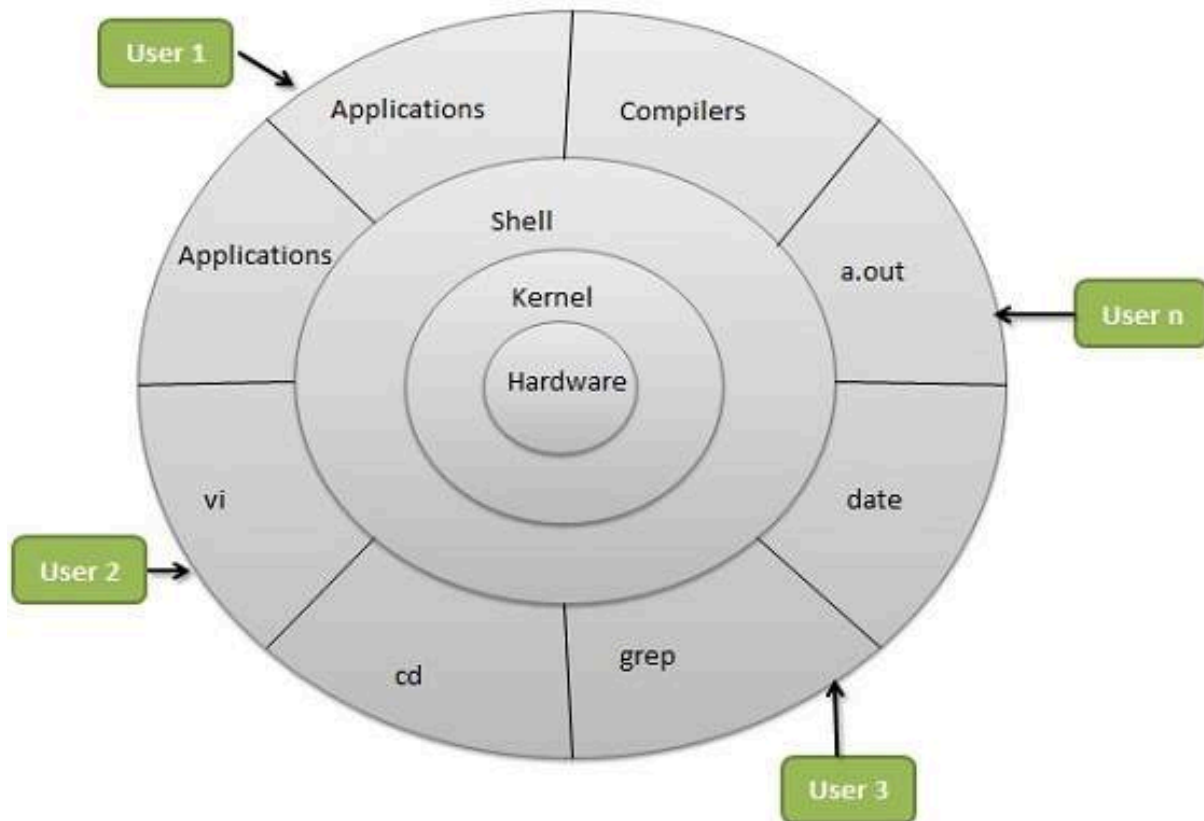
1.3 Memes

This SSD is Super slow and unreliable!

Me running my OS on a USB stick



2. Shell



2.1 What is a Shell?

A shell is a special user program that provides an interface for the user to use operating system services. Informally, It is a command-line interpreter that bridges the user and the operating system kernel.

Examples →

- **bash** (Bourne Again Shell) → One of the most common and widely used shells in Unix-like systems (e.g., Linux) . It is known for its scripting capabilities and built-in features.
- **sh** (Bourne Shell) → The original Unix shell created by Stephen Bourne. It is simpler than Bash and often serves as the standard shell in scripts.
- **csh** (C Shell) → Shell with syntax resembling the C programming language. It includes features like history and job control.
- **zsh** (Z Shell) → An extended version of Bash with additional features like better theme support, improved autocompletion, and more.
- **ksh** (Korn Shell) → A shell designed to combine the best features of the Bourne and C Shells.

2.2 Core Concepts

- **Commands** → Instructions to perform tasks.
 - **Environment Variables** → Store system/session data.
 - **Redirection** → Control input/output streams.
 - **Pipelines** → Connect multiple commands.
-

3. Terminal

3.1 What is a Terminal?

A terminal (also known as terminal emulator) is a software application that provides a graphical interface for interacting with the shell.

A program which is responsible for providing an interface to a user so that he/she can access the shell. It basically allows users to enter commands and see the output of those commands in a text based interface.

3.2 Terminal Shortcuts

- **CTRL+C** → Interrupt running process
- **CTRL+Z** → Suspend running process
- **TAB** → Autocomplete commands/paths
- **↑ / ↓** → Command history navigation

3.3 Multiplexing & Sessions

- **tmux** → Terminal multiplexer for multiple sessions in one terminal.
-

4. Shell Commands

4.1 File Navigation

```
ls      # list files
tree    # visualize folder hierarchy
```

```
pwd    # print working directory
cd     # change directory
```

4.2 File Operations

```
touch file.c    # create file
mkdir lab       # create folder
cp file1 file2  # copy
mv file1 file2  # rename/move
rm file.c       # delete
```

4.3 Compilation and Execution

```
gcc hello.c -o hello # compile
./hello              # run the binary
clear                # clean terminal
time ./a.out         # benchmark execution
```

4.4 Permissions and Ownership

```
chmod +x script.sh # make script executable
ls -l              # show permissions
```

4.5 Searching and Filtering

```
grep "main" file.c    # search for text
find . -name "*.c"     # find files
cat , head , tail     # view contents
```

4.6 Process and System Monitoring

```
top    # live process monitor
ps aux # list processes
```

```
kill -9 <pid> # forcefully kill
```

4.7 Common Commands

```
echo "Hello, World!" # Print text
whoami               # Show current user
uname -a             # System info
ps aux               # Show running processes
kill PID             # Terminate a process
```

4.8 Redirection & Pipes

```
command > file.txt # Redirect stdout to file
command >> file.txt # Append stdout to file
command 2> err.txt # Redirect stderr
cmd1 | cmd2        # Pipe output of cmd1 to cmd2
```

4.9 Predefined Keywords

```
echo "User: $USER"
echo "Date: $(date)"
echo "Formatted Date: $(date '+%Y-%m-%d_%H-%M-%S')"
```